9

# Programação funcional

Universidade Estadual da Paraíba (UEPB)
Paradigmas de Programação
Kláudio Henrique Mascarenhas Medeiros

# Paradigma funcional

- No paradigma funcional, a computação envolve funções matemáticas, evitando assim o uso de estados ou dados cujos valores são alterados durante o programa (comum na programação imperativa).

# Paradigma funcional

- No **paradigma funcional**, a computação envolve funções matemáticas, evitando assim o uso de estados ou dados cujos valores são alterados durante o programa (comum na programação imperativa).

- O programa então é composto por essas funções que resolvem um determinado problema.

# Paradigma funcional (funções)

- Funções mapeiam elementos do conjunto domínio para elementos de um contradomínio.

# Paradigma funcional (funções)

- Funções mapeiam **elementos do conjunto domínio para elementos de um contradomínio**.

- Os elementos do contradomínio alcançados por algum do domínio pela função formam o **conjunto imagem**.

# Paradigma funcional (funções)

- Funções mapeiam **elementos do conjunto domínio para elementos de um contradomínio**.

- Os elementos do contradomínio alcançados por algum do domínio pela função formam o **conjunto imagem**.

- A **ordem de avaliação** é controlada por expressões condicionais e por recursão (não por iteração)

# Paradigma funcional (funções)

- Funções mapeiam **elementos do conjunto domínio para elementos de um contradomínio**.
- Os elementos do contradomínio alcançados por algum do domínio pela função formam o **conjunto imagem**.
- A **ordem de avaliação** é controlada por expressões condicionais e por recursão (não por iteração)
- **Não possuem efeitos colaterais!** (mais adiante)

# Paradigma funcional (funções)

- Uma função é definida através de definição de um **nome, seus parâmetros e qual a sua expressão** (e.g. square(x) x * x).

# Paradigma funcional (funções)

- Uma função é definida através de definição de um **nome, seus parâmetros e qual a sua expressão** (e.g. square(x) x * x).

- A partir do nome da função e informação sobre elementos do domínio, obtemos um da **imagem** (e.g. square(2.0) = 4.0).

# Paradigma funcional (funções)

- Uma função é definida através de definição de um **nome, seus parâmetros e qual a sua expressão** (e.g. square(x) x * x).

- A partir do nome da função e informação sobre elementos do domínio, obtemos um da **imagem** (e.g. square(2.0) = 4.0).

- Pode-se utilizar notação **lambda**, que define-se uma função sem nome (e.g. λ(x) x * x)

# Paradigma funcional (funções)

```
nums = input("Escreva números separados por espaço: ")

print("Os números ao quadrado:")
print(list(map(lambda x: int(x) ** 2, nums.split(" "))))
```

- É criado uma função lambda de um parâmetro que converte o argumento em inteiro e o eleva ao quadrado. Essa função é **anônima**.

# Paradigma funcional (funções)

```
nums = input("Escreva números separados por espaço: ")

print("Os números ao quadrado:")
print(list(map(lambda x: int(x) ** 2, nums.split(" "))))
```

- Essa expressão acima pode ser uma expressão necessária apenas neste ponto. Pode não fazer tanto sentido definir uma função nomeada para isso.

# Paradigma funcional

- No paradigma funcional, o **parâmetro da função** representa um elemento qualquer existente no domínio dessa função.

# Paradigma funcional

- No paradigma funcional, o **parâmetro da função** representa um elemento qualquer existente no domínio dessa função.

- Durante a avaliação, esse parâmetro é vinculado a um membro em particular. Durante a avaliação, o valor é tratado como uma constante.

# Paradigma funcional

- No paradigma funcional, o **parâmetro da função** representa um elemento qualquer existente no domínio dessa função.

- Durante a avaliação, esse parâmetro é vinculado a um membro em particular. Durante a avaliação, o valor é tratado como uma constante.

- Quando fornecidos os mesmos argumentos, as funções sempre produzem o mesmo resultado.

# Paradigma funcional

- Nesse paradigma, estamos constantemente preocupados em **que funções podemos utilizar para converter as entradas fornecidas nas saídas desejadas.**

# Paradigma funcional

- Nesse paradigma, estamos constantemente preocupados em **que funções podemos utilizar para converter as entradas fornecidas nas saídas desejadas.**

- A sintaxe é adequada para definição de **funções compostas** (funções dentro de funções). Esse é um contraponto ao paradigma imperativo, que trata de sequências de comandos (e.g. f2(f1(x)) ).

# Paradigma funcional

- Exemplo de composição de funções
  - square(x) x * x
  - apply_to_all(square, (1, 2, 3))

# Paradigma funcional

- Exemplo de composição de funções
  - square(x) x * x
  - apply_to_all(square, (1, 2, 3))

- O resultado acima será (1, 4, 9).

# Paradigma funcional

- Exemplo de composição de funções
  - square(x) x * x
  - apply_to_all(square, (1, 2, 3))

- O resultado acima será (1, 4, 9).

- É semelhante ao map de Python, por exemplo. A instrução map(f, i) aplica a função f a cada elemento do iterável i.

# Paradigma funcional

```haskell
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9]

filter_pairs :: [Int] -> [Int]
filter_pairs l = filter even l

sum_numbers :: [Int] -> Int
sum_numbers l = foldl (+) 0 l

filter_sum :: [Int] -> Int
filter_sum l = (sum_numbers . filter_pairs) l

main :: IO ()
main = print(filter_sum nums)
```

```
[1 of 1] Compiling Main
Linking main ...
20
```

# Paradigma funcional (funções)

```haskell
main :: IO ()
main = print (zipWith (+) [2, 3, 4] [5, 6, 7]) >>
       print (zipWith (^) [2, 3, 4] [5, 6]) >>
```

```
[1 of 1] Compiling Main
Linking main ...
[7,9,11]
[32,729]
```

- Em Haskell, o zipWith junta operandos de duas listas diferentes, aplicando uma operação entre eles

# Paradigma funcional

- Algumas características do paradigma funcional.
    - não existem variáveis e sim **expressões**

# Paradigma funcional

```
1  num = 10
2  num = 20
3  main = print(num)
```

```
[1 of 1] Compiling Main
main.hs:2:1: error:
    Multiple declarations of `num'
    Declared at: main.hs:1:1
                 main.hs:2:1
  |
2 | num = 20
  | ^^^
```

# Paradigma funcional

- Algumas características do paradigma funcional.
  - não existem variáveis e sim **expressões**
  - não existem comandos e sim **funções**

# Paradigma funcional

```haskell
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9]

filter_pairs :: [Int] -> [Int]
filter_pairs l = filter even l

sum_numbers :: [Int] -> Int
sum_numbers l = foldl (+) 0 l

filter_sum :: [Int] -> Int
filter_sum l = (sum_numbers . filter_pairs) l

main :: IO ()
main = print(filter_sum nums)
```

```
[1 of 1] Compiling Main
Linking main ...
20
```

# Paradigma funcional

- Algumas características do paradigma funcional.
  - não existem variáveis e sim **expressões**
  - não existem comandos e sim **funções**
  - não existem efeitos colaterais

# Paradigma funcional

- Algumas características do paradigma funcional.
  - não existem variáveis e sim **expressões**
  - não existem comandos e sim **funções**
  - não existem efeitos colaterais
  - fluxo de controle dado por **exp. cond. e recursão**

# Paradigma funcional

```haskell
fibonacci 0 = 0
fibonacci 1 = 1
fibonacci n = fibonacci(n - 1) + fibonacci(n - 2)


main :: IO ()
main = print(fibonacci(5))
```

```
[1 of 1] Compiling Main
Linking main ...
5
```

# Paradigma funcional

- Algumas características do paradigma funcional.
  - não existem variáveis e sim **expressões**
  - não existem comandos e sim **funções**
  - não existem efeitos colaterais
  - fluxo de controle dado por **exp. cond. e recursão**
  - **não há armazenamento**
    - *lazy evaluation*, recursão, função de alta ordem

# Paradigma funcional

Como poderia ser feita uma estrutura de repetição sem a utilização de variáveis?

# Paradigma funcional

- As funções mapeiam valores de um tipo para valores de outro tipo (e.g. isDigit :: Char -> Bool para uma função isDigit que mapeia um Char para um Bool).

# Paradigma funcional

- As funções mapeiam valores de um tipo para valores de outro tipo (e.g. isDigit :: Char -> Bool para uma função isDigit que mapeia um Char para um Bool).

- Pode acontecer da função ter mais de um parâmetro (e.g. add :: (Int, Int) -> Int)

# Paradigma funcional

- As funções mapeiam valores de um tipo para valores de outro tipo (e.g. isDigit :: Char -> Bool para uma função isDigit que mapeia um Char para um Bool).

- Pode acontecer da função ter mais de um parâmetro (e.g. add :: (Int, Int) -> Int)

- As funções podem ter um argumento:
    - e.g. pi = 3.14, circumf r = 2 * pi * r
    - area rad = pi * rad ^2

# Paradigma funcional

- Exemplo: como calcular a soma de todos os números entre 0 e 100.

- Como resolver isso de forma iterativa (e.g. paradigma imperativo)? Como resolver isso de outro modo, suportado pelo paradigma funcional?

# Paradigma funcional

- Exemplo: como calcular a soma de todos os números entre 0 e 100 (Haskell)

```
soma[] = 0
soma(x:xs) = x + soma xs
> prelude soma[1..100]
```

A solução em Haskell para esse problema envolve recursividade e casamento de padrões

Perceba a existência de um caso base e de um caso onde há a chamada à própria função. Aqui, resolvemos o problema recursivamente.

Nada está sendo de fato armazenado (e.g. uma variável com a soma).

No exemplo, perceba que a chamada recursiva é a última expressão da função soma. No caso, ela é uma recursão em cauda (*tail recursion*).

# Paradigma funcional

- As linguagens funcionais costumam utilizar o conceito de cabeça (*head*) e cauda (*tail*).

# Paradigma funcional

- As linguagens funcionais costumam utilizar o conceito de cabeça (*head*) e cauda (*tail*).

- O primeiro elemento de uma lista, por exemplo, muitas vezes é acessado através de **car**. O restante da lista é acessado através de **cdr** (lê-se “coulder”).

# Paradigma funcional

- As linguagens funcionais costumam utilizar o conceito de cabeça (*head*) e cauda (*tail*).

- O primeiro elemento de uma lista, por exemplo, muitas vezes é acessado através de **car**. O restante da lista é acessado através de **cdr** (lê-se “coulder”).

- Essas palavras são utilizadas em algumas linguagens de programação, como Common LISP e Scheme.

# Paradigma funcional (pseudo-código)

```
lista1 = list(1, 2, 3, 4, 5)
car lista1
> 1
cdr lista1
> (2, 3, 4)
car (cdr lista1)
> 2
cadr lista1
> 2
caddr lista1
> # Qual seria o valor aqui?
```

# Paradigma funcional (pseudo-código)

```
lista1 = list(1, 2, 3, 4, 5)
car lista1
> 1
cdr lista1
> (2, 3, 4)
car (cdr lista1)
> 2
cadr lista1
> 2
caddr lista1
> 3
```

# Paradigma funcional

```haskell
filter_sum :: [Int] -> Int

filter_sum [] = 0
filter_sum (x:xs)
    | even x = x + filter_sum xs
    | otherwise = filter_sum xs


nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

main :: IO ()
main = print(filter_sum nums)
```

```
[1 of 1] Compiling Main
Linking main ...
30
```

# Paradigma funcional (Haskell)

- Haskell possui algumas características que são relacionadas com o paradigma funcional:
    - *lazy evaluation* (avaliação preguiçosa)
    - funções são valores de primeira classe
    - funções e tipos paramétricos

# Paradigma funcional (Haskell)

- A **avaliação preguiçosa** (ou *lazy evaluation*) é uma estratégia na qual a avaliação de uma expressão só acontece quando o resultado é realmente necessário.

# Paradigma funcional (Haskell)

- A **avaliação preguiçosa** (ou *lazy evaluation*) é uma estratégia na qual a avaliação de uma expressão só acontece quando o resultado é realmente necessário.

- e.g. “x = 3 + 5”:
  - algumas linguagens avaliam a expressão 3 + 5 e, então, atribuem (x = 8) (*eager evaluation*).

# Paradigma funcional (Haskell)

- A **avaliação preguiçosa** (ou *lazy evaluation*) é uma estratégia na qual a avaliação de uma expressão só acontece quando o resultado é realmente necessário.

- e.g. “x = 3 + 5”:
  - algumas linguagens avaliam a expressão 3 + 5 e, então, atribuem (x = 8) (*eager evaluation*).
  - outras linguagens não avaliam o 3 + 5, só fazendo isso quando o valor da expressão for realmente preciso (*lazy evaluation*)

# Paradigma funcional (*lazy evaluation*)

- Quando a expressão é avaliada, ele é mantido (*memoized*) de forma que não precisa ser recalculado.

Link: http://blog.haskellbr.com/2016/01/17/lazy-evaluation-em-haskell.html

# Paradigma funcional (*lazy evaluation*)

- Quando a expressão é avaliada, ele é mantido (*memoized*) de forma que não precisa ser recalculado.

- No exemplo anterior, quando calcularmos que x é igual a 8, não precisamos mais recalcular.

Link: http://blog.haskellbr.com/2016/01/17/lazy-evaluation-em-haskell.html

# Paradigma funcional (*lazy evaluation*)

- Quando a expressão é avaliada, ele é mantido (*memoized*) de forma que não precisa ser recalculado.

- No exemplo anterior, quando calcularmos que x é igual a 8, não precisamos mais recalcular.

- Exemplo: listas em Haskell como [1..]. Essa stream contém todos os inteiros positivos, sendo uma lista infinita, avaliada de forma preguiçosa.

Link: http://blog.haskellbr.com/2016/01/17/lazy-evaluation-em-haskell.html

# Paradigma funcional (ex. em Python)

```
import sys

lista_lazy = range(1, 20, 2)
lista_lazy2 = range(1, 1000, 2)
print(lista_lazy)
lista_total = [i for i in range(1, 20, 2)]
print(lista_total)

print("Comparando duas listas: uma lazy e uma completa")
print(sys.getsizeof(lista_lazy))
print(sys.getsizeof(lista_total))

print()

print("Comparando duas listas de tamanhos diferentes com lazy")
print(sys.getsizeof(lista_lazy))
print(sys.getsizeof(lista_lazy2))
```

# List comprehension

- *List comprehension* é uma maneira de criar listas baseadas em listas existentes.

# List comprehension

- *List comprehension* é uma maneira de criar listas baseadas em listas existentes.

- Elas dependem de uma expressão (que define o que será incluído) e uma lista (a partir de qual a nova lista é baseada).

# List comprehension

- *List comprehension* é uma maneira de criar listas baseadas em listas existentes.

- Elas dependem de uma expressão (que define o que será incluído) e uma lista (a partir de qual a nova lista é baseada).

- Essas listas por vezes parecem com listas que definimos na matemática (e.g. $\{2x \mid x \in \mathbb{R},\ x > 10\}$).

# List comprehension

```
lista1 = ["klaus", "stella", "michel", "candy"]

lista2 = [elem for elem in lista1 if "a" in elem]
print(lista2)

lista3 = [elem.upper() for elem in lista1]
print(lista3)

# Idêntico à lista2
lista4 = list(filter(lambda x: "a" in x, lista1))
print(lista4)

# Idêntico à lista3
lista5 = list(map(lambda x: x.upper(), lista1))
print(lista5)
```

# Paradigma funcional (Haskell)

- A função ser um valor de primeira classe indica que ela é tão relevante quanto outros tipos, como inteiros ou outros tipos da linguagem.

# Paradigma funcional (Haskell)

- A função ser um valor de primeira classe indica que ela é tão relevante quanto outros tipos, como inteiros ou outros tipos da linguagem.

- Isso significa que funções podem ser passadas como argumento para outras funções, retornadas por funções e constituírem estruturas de dados!

# Paradigma funcional (Haskell)

- A função ser um valor de primeira classe indica que ela é tão relevante quanto outros tipos, como inteiros ou outros tipos da linguagem.

- Isso significa que funções podem ser passadas como argumento para outras funções, retornadas por funções e constituírem estruturas de dados!

- Ex: map(x -> 2 * x) [1, 2, 3] resulta [2, 4, 6]

# Paradigma funcional

- Algumas linguagens funcionais suportam apenas funções unárias (i.e. com um parâmetro).

# Paradigma funcional

- Algumas linguagens funcionais suportam apenas funções unárias (i.e. com um parâmetro).

- Caso se deseje criar uma função com mais de um parâmetro, pode-se utilizar o *currying*.

# Paradigma funcional

- Algumas linguagens funcionais suportam apenas funções unárias (i.e. com um parâmetro).

- Caso se deseje criar uma função com mais de um parâmetro, pode-se utilizar o ***currying***.

- O *currying* consiste em transformar uma função de vários parâmetros em funções de um parâmetro, chamadas sequencialmente.

# Paradigma funcional

```
function soma(n1, n2){
    return n1 + n2;
}

function soma_curry(n1){
    return function (n2){
        return n1 + n2;
    }
}

console.log(soma_curry(2)(4))
```

```
node /tmp/d8WpkcSeER.js
6
```

# Paradigma funcional

- Como vantagens, o paradigma funcional simplifica a solução de alguns tipos de problemas (e.g. prova de propriedades).

# Paradigma funcional

- Como vantagens, o paradigma funcional simplifica a solução de alguns tipos de problemas (e.g. prova de propriedades).

- Porém, podem apresentar soluções ineficientes, além de utilizar mecanismos muito simples de entrada e saída.

# Paradigma funcional

- Como vantagens, o paradigma funcional simplifica a solução de alguns tipos de problemas (e.g. prova de propriedades).

- Porém, podem apresentar soluções ineficientes, além de utilizar mecanismos muito simples de entrada e saída.

- Por fim, o mundo não é modelado de maneira funcional.

# Referências

SEBESTA, Robert M. **Conceitos de Linguagens de Programação**. Bookman, 2018.